

Chapter 20: HttpClient

1. Overview

`HttpClient` is Angular's built-in service for communicating with backend servers over HTTP. It wraps the browser's `XMLHttpRequest` (or, since Angular 17+, an optional `fetch()`-based backend) in an RxJS `observable`-based API, giving you typed responses, automatic JSON parsing, a composable interceptor pipeline, request/response transformation, progress events, and first-class testability via `HttpTestingController`.

Every interview on Angular services eventually touches `HttpClient` because it sits at the intersection of several core Angular/RxJS concepts:

- Dependency injection (`provideHttpClient()`, `HttpClientModule`, injector-based interceptors)
- RxJS observables (cold streams, operators, cancellation via `unsubscribe`)
- Generics and typed APIs
- Error handling patterns (`catchError`, `retry`, `throwError`)
- Testing strategy (mocking network calls deterministically)

This chapter treats `HttpClient` as both an API surface and an RxJS pipeline you need to reason about precisely — not just "call `.get()` and subscribe."

2. Core Concepts

2.1 Setting up HttpClient — `provideHttpClient()` vs `HttpClientModule`

Standalone (modern, Angular 15+ preferred, mandatory mental model going forward):

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideHttpClient, withInterceptors, withFetch } from '@angular/common/http';
import { AppComponent } from './app/app.component';
import { authInterceptor } from './app/interceptors/auth.interceptor';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(
      withInterceptors([authInterceptor]), // functional interceptors, run in array order
      withFetch(),                       // opt into the fetch()-based backend
      // withInterceptorsFromDi(),        // bridge to legacy class-based DI interceptors
      // withJsonSupport(),
      // withXsrfConfiguration({ cookieName: 'XSRF-TOKEN', headerName: 'X-XSRF-TOKEN' }),
    ),
  ],
});
```

NgModule-based (legacy but still common in existing codebases):

```
@NgModule({
  imports: [HttpClientModule],
})
export class AppModule {}
```

Key interview point: `provideHttpClient()` is a **tree-shakable provider function**, not a module. It composes "features" via `with*()` functions — this is Angular's modern pattern (same idea as `provideRouter(withPreloading(...))`). If you don't call `provideHttpClient()` (or import `HttpClientModule`) anywhere in the injector tree, injecting `HttpClient` throws `NullInjectorError: No provider for HttpClient`.

2.2 The fetch-based backend (`withFetch()`)

Historically `HttpClient` always used `HttpXhrBackend`, built on `XMLHttpRequest`. Since Angular 17, `withFetch()` swaps in `FetchBackend`, built on the native `fetch()` API.

Why it matters:

- **SSR / Node environments:** `fetch` is natively available in Node 18+, avoiding XHR polyfills during server-side rendering — reduces bundle size and aligns with platform-native APIs.
- **Streaming:** fetch supports streaming response bodies more naturally.
- Trade-off: `fetch()` has **no native upload-progress events** — Angular's `FetchBackend` still reports download progress, but upload progress reporting is limited/unsupported in some browsers compared to XHR's `upload.onprogress`. If you need reliable **upload** progress tracking (e.g., file upload with a progress bar), the XHR backend is still the safer choice.
- Both backends implement the same `HttpBackend` interface, so switching is a provider-config change, not an application code change.

2.3 Making requests — GET/POST/PUT/PATCH/DELETE

```
this.http.get<User[]>('/api/users');
this.http.post<User>('/api/users', newUser);
this.http.put<User>(`/api/users/${id}`, updatedUser);
this.http.patch<User>(`/api/users/${id}`, partialUser);
this.http.delete<void>(`/api/users/${id}`);
```

Every method returns `Observable<T>` where `T` is a **generic type parameter you supply** — `HttpClient` does **not** validate the response against `T` at runtime. It's a compile-time-only type assertion layered on top of `JSON.parse()`'s `any`. This is a classic interview trap: "does `HttpClient` validate my interface at runtime?" — **No**. If the server sends a different shape, you get a type-safe-looking object that silently violates your interface (runtime type unsafety hiding behind a generic).

2.4 `HttpParams` — building query strings immutably

```
import { HttpParams } from '@angular/common/http';

let params = new HttpParams()
  .set('page', page.toString())
  .set('pageSize', pageSize.toString());

if (search) {
  params = params.set('q', search);
}

this.http.get<PagedResult<User>>('/api/users', { params });
```

`HttpParams` instances are **immutable** — `.set()` / `.append()` / `.delete()` return a *new* instance rather than mutating in place (same design philosophy as `HttpHeaders`, and analogous to how `Immutable.js` or persistent data structures work). This is why the pattern above reassigns `params =` rather than calling `params.set(...)` and discarding the result.

Shortcut using `fromObject`:

```
const params = new HttpParams({ fromObject: { page, pageSize, q: search ?? '' } });
```

2.5 `HttpHeaders` — immutable header construction

```
import { HttpHeaders } from '@angular/common/http';

const headers = new HttpHeaders()
  .set('Content-Type', 'application/json')
  .set('Authorization', `Bearer ${token}`);

this.http.post('/api/orders', order, { headers });
```

Same immutability rule as `HttpParams`. Note: you almost never need to manually set `Content-Type: application/json` — `HttpClient` infers and sets it automatically when the request body is a plain JS object.

2.6 `observe` option — response, body, or events

The `observe` option controls what shape the returned `Observable` emits:

<code>observe</code> value	Emits	Use case
<code>'body'</code> (default)	Just the parsed body, typed as <code>T</code>	Normal case — you only need the data
<code>'response'</code>	Full <code>HttpResponse<T></code> (status, headers, body)	Need status code, headers, or need to distinguish 204 No Content from an empty body

observe value	Emits	Use case
'events'	Stream of <code>HttpEvent<T></code> (Sent, UploadProgress, DownloadProgress, ResponseHeader, Response, User)	Progress bars, fine-grained lifecycle tracking

```
this.http.get<User>('/api/users/1', { observe: 'response' }).subscribe(res => {
  console.log(res.status);           // 200
  console.log(res.headers.get('X-Total-Count'));
  console.log(res.body);             // User | null
});
```

2.7 reportProgress + observe: 'events' for progress tracking

```
import { HttpEventType, HttpRequest } from '@angular/common/http';

const req = new HttpRequest('POST', '/api/upload', formData, {
  reportProgress: true,
});

this.http.request(req).subscribe(event => {
  switch (event.type) {
    case HttpEventType.Sent:
      console.log('Request sent');
      break;
    case HttpEventType.UploadProgress:
      const pct = Math.round((100 * event.loaded) / (event.total ?? event.loaded));
      console.log(`Uploaded ${pct}%`);
      break;
    case HttpEventType.DownloadProgress:
      console.log(`Downloaded ${event.loaded} bytes`);
      break;
    case HttpEventType.Response:
      console.log('Done', event.body);
      break;
  }
});
```

`reportProgress: true` must be explicitly set — by default Angular does not emit progress events, since computing them has overhead and most requests don't need it. Without `reportProgress: true`, you only ever get the final `HttpEventType.Response` event even with `observe: 'events'`.

2.8 responseType

Controls how the raw response body is parsed: `'json'` (default), `'text'`, `'blob'`, `'arraybuffer'`. Needed for file downloads (`'blob'`) or plain-text endpoints.

```
this.http.get('/api/report.pdf', { responseType: 'blob' });
```

2.9 Error handling — `catchError`, `HttpErrorResponse`, `retry`

Every failed request (network failure, non-2xx status, or a client-side error such as a JSON parse failure) surfaces as an `HttpErrorResponse` emitted through the Observable's **error channel**, not its next channel.

```
import { catchError, retry, throwError, timeout } from 'rxjs';
import { HttpErrorResponse } from '@angular/common/http';

getUsers(): Observable<User[]> {
  return this.http.get<User[]>('/api/users').pipe(
    timeout(5000),
    retry({ count: 2, delay: 1000 }),
    catchError((err: HttpErrorResponse) => {
      if (err.status === 0) {
        // Network-level error, CORS failure, or client-side ProgressEvent – no response
        // reached the app.
        console.error('Network error', err.error);
      } else {
        console.error(`Server returned ${err.status}: ${err.error?.message}`);
      }
      return throwError(() => new Error('Failed to load users; please retry.'));
    })
  );
}
```

Key structural facts about `HttpErrorResponse`:

- `error.status` — HTTP status code, or `0` for a network-level failure (no HTTP response at all — DNS failure, CORS block, offline).
- `error.error` — the parsed response body (if JSON) or an `ErrorEvent` / `ProgressEvent` for client-side/network errors. Shape is inconsistent between server errors and network errors — always guard against both.
- `error.message` — a generic human-readable summary Angular constructs, rarely what you want to show users directly.
- `error.headers`, `error.url`, `error.ok` — also available.

2.10 Request cancellation

Because `HttpClient` observables are **cold**, cancellation is just `unsubscribe()` — calling it **aborts the underlying XHR/fetch request**, not merely stops delivering values to your callback.

```
const sub = this.http.get('/api/search?q=foo').subscribe(...);
// user navigates away / types a new character
sub.unsubscribe(); // actually calls xhr.abort() under the hood
```

In practice you rarely call `unsubscribe()` manually — instead you let RxJS operators do it for you:

```
searchTerm$.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term => this.http.get<Result[]>(`/api/search?q=${term}`))
).subscribe(results => this.results = results);
```

`switchMap` cancels the previous inner HTTP observable whenever a new source value arrives — this is the standard idiom for "cancel in-flight request when a newer one supersedes it" (type-ahead search, autocomplete).

The `async` pipe and `takeUntilDestroyed()` are the two most common auto-unsubscribe mechanisms in components, avoiding both memory leaks and dangling in-flight requests.

2.11 Interceptors — the pipeline `HttpClient` calls run through

Functional interceptors (Angular 15+, standalone-first API):

```
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token;
  const cloned = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;
  return next(cloned);
};
```

Registered via `provideHttpClient(withInterceptors([authInterceptor, loggingInterceptor]))`. Interceptors run in the order given, wrapping each other like middleware — each must call `next(req)` to continue the chain, or return its own `Observable<HttpEvent<any>>` to short-circuit it (e.g., serving from cache).

`HttpRequest` and `HttpHeaders` / `HttpParams` are all **immutable** — an interceptor must produce a new request via `req.clone({...})` rather than mutating `req` directly.

2.12 Testing with `HttpTestingController`

`HttpClientTestingModule` (or `provideHttpClientTestingModule()` in standalone tests) replaces the real backend with `HttpTestingController`, which lets tests assert exactly what requests were made and flush canned responses synchronously — no real network, no timers, fully deterministic.

3. Code Examples

3.1 Typed CRUD service with generics, params, headers, and error handling

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams, HttpResponseError } from '@angular/common/http';
import { Observable, catchError, throwError } from 'rxjs';
```

```

export interface User {
  id: number;
  name: string;
  email: string;
}

export interface PagedResult<T> {
  items: T[];
  total: number;
}

@Injectable({ providedIn: 'root' })
export class UserService {
  private readonly http = inject(HttpClient);
  private readonly baseUrl = '/api/users';

  getUsers(page: number, pageSize: number, search?: string): Observable<PagedResult<User>> {
    let params = new HttpParams()
      .set('page', page)
      .set('pageSize', pageSize);

    if (search) {
      params = params.set('q', search);
    }

    return this.http
      .get<PagedResult<User>>(this.baseUrl, { params })
      .pipe(catchError(this.handleError));
  }

  getUser(id: number): Observable<User> {
    return this.http
      .get<User>(`${this.baseUrl}/${id}`)
      .pipe(catchError(this.handleError));
  }

  createUser(user: Omit<User, 'id'>): Observable<User> {
    return this.http
      .post<User>(this.baseUrl, user)
      .pipe(catchError(this.handleError));
  }

  updateUser(id: number, changes: Partial<User>): Observable<User> {
    return this.http
      .patch<User>(`${this.baseUrl}/${id}`, changes)
      .pipe(catchError(this.handleError));
  }

  deleteUser(id: number): Observable<void> {
    return this.http
      .delete<void>(`${this.baseUrl}/${id}`)
      .pipe(catchError(this.handleError));
  }
}

```

```

}

private handleError(error: HttpErrorResponse) {
  const message =
    error.status === 0
      ? 'Network error – check your connection.'
      : `Server error ${error.status}: ${error.error?.message ?? error.statusText}`;
  return throwError(() => new Error(message));
}
}

```

3.2 File upload with progress tracking

```

import { Injectable, inject } from '@angular/core';
import {
  HttpClient,
  HttpEventType,
  HttpRequest,
  HttpErrorResponse,
} from '@angular/common/http';
import { Observable, catchError, throwError } from 'rxjs';

export interface UploadProgress {
  percent: number;
  done: boolean;
}

@Injectable({ providedIn: 'root' })
export class FileUploadService {
  private readonly http = inject(HttpClient);

  upload(file: File): Observable<UploadProgress> {
    const formData = new FormData();
    formData.append('file', file, file.name);

    const req = new HttpRequest('POST', '/api/upload', formData, {
      reportProgress: true,
    });

    return new Observable<UploadProgress>(observer => {
      const sub = this.http.request(req).pipe(
        catchError((err: HttpErrorResponse) => {
          observer.error(err);
          return throwError(() => err);
        })
      ).subscribe(event => {
        if (event.type === HttpEventType.UploadProgress && event.total) {
          observer.next({
            percent: Math.round((100 * event.loaded) / event.total),
            done: false,
          });
        } else if (event.type === HttpEventType.Response) {

```



```

        observer.next({ percent: 100, done: true });
        observer.complete();
    }
});

// Cancellation: unsubscribing from the returned Observable aborts the XHR.
return () => sub.unsubscribe();
});
}
}

```

Component usage:

```

onFileSelected(file: File) {
    this.uploadService.upload(file).subscribe({
        next: progress => (this.uploadPercent = progress.percent),
        error: err => (this.uploadError = err.message),
        complete: () => (this.uploadDone = true),
    });
}

```

3.3 Unit test with `HttpTestingController`

```

import { TestBed } from '@angular/core/testing';
import {
    HttpTestingController,
    provideHttpClientTesting,
} from '@angular/common/http/testing';
import { provideHttpClient } from '@angular/common/http';
import { UserService, User, PagedResult } from './user.service';

describe('UserService', () => {
    let service: UserService;
    let httpMock: HttpTestingController;

    beforeEach(() => {
        TestBed.configureTestingModule({
            providers: [
                UserService,
                provideHttpClient(),
                provideHttpClientTesting(),
            ],
        });

        service = TestBed.inject(UserService);
        httpMock = TestBed.inject(HttpTestingController);
    });

    afterEach(() => {
        // Fails the test if any request was expected but never made, or made but never
        // asserted.
        httpMock.verify();
    });
});

```

```

});

it('fetches a page of users with correct query params', () => {
  const mockResult: PagedResult<User> = {
    items: [{ id: 1, name: 'Ada', email: 'ada@example.com' }],
    total: 1,
  };

  service.getUsers(1, 20, 'ada').subscribe(result => {
    expect(result).toEqual(mockResult);
  });

  const req = httpMock.expectOne(
    r => r.url === '/api/users' && r.params.get('q') === 'ada'
  );

  expect(req.request.method).toBe('GET');
  expect(req.request.params.get('page')).toBe('1');
  expect(req.request.params.get('pageSize')).toBe('20');

  req.flush(mockResult);
});

it('surfaces a friendly error on 500', () => {
  let capturedError: Error | undefined;

  service.getUser(99).subscribe({
    next: () => fail('expected an error'),
    error: err => (capturedError = err),
  });

  const req = httpMock.expectOne('/api/users/99');
  req.flush(
    { message: 'User table unavailable' },
    { status: 500, statusText: 'Internal Server Error' }
  );

  expect(capturedError?.message).toContain('Server error 500');
  expect(capturedError?.message).toContain('User table unavailable');
});

it('handles a network error (status 0)', () => {
  let capturedError: Error | undefined;

  service.getUser(1).subscribe({
    next: () => fail('expected an error'),
    error: err => (capturedError = err),
  });

  const req = httpMock.expectOne('/api/users/1');
  req.error(new ProgressEvent('network error'), { status: 0 });

  expect(capturedError?.message).toBe('Network error – check your connection.');
```

```
});

it('sends a DELETE request', () => {
  service.deleteUser(1).subscribe();

  const req = httpMock.expectOne('/api/users/1');
  expect(req.request.method).toBe('DELETE');
  req.flush(null);
});
});
```

Key API points on `HttpTestingController`:

- `expectOne(matcher)` — asserts exactly one matching request exists, and returns a `TestRequest` handle. Throws if zero or more than one match.
- `TestRequest.flush(body, opts?)` — resolves the request synchronously with a fake response body/status.
- `TestRequest.error(errorEvent, opts?)` — simulates a network-level failure.
- `httpMock.verify()` — call in `afterEach` to catch requests you forgot to flush/expect; a leaked, unflushed request is a real bug (component stuck in a loading state) that `verify()` surfaces immediately.

4. Internal Working

4.1 The pipeline: `HttpClient` → `HttpInterceptorHandler` → `interceptor chain` → `HttpBackend`

When you call `this.http.get(url)`:

1. `HttpClient` constructs an `HttpRequest` object (immutable value object describing method, url, body, headers, params, and options).
2. `HttpClient.request()` builds an RxJS `Observable` via `new Observable(observer => {...})` — nothing executes yet. This is why `HttpClient` calls are **cold**: no work happens until something subscribes.
3. On subscription, the request is handed to an `HttpHandler` — specifically the head of a chain built from your registered interceptors. Each functional interceptor is composed into a chain resembling middleware: `interceptor1(req, (req2) => interceptor2(req2, (req3) => ... backendCall(reqN)))`.
4. Each interceptor may transform the request (via `req.clone()`), short-circuit with its own observable (e.g., cache hit), or call `next(req)` to forward to the next link.
5. At the end of the chain sits the true `HttpBackend` implementation — `HttpXhrBackend` (XHR) or `FetchBackend` (fetch), selected based on whether `withFetch()` was configured.
6. The backend performs the actual `XMLHttpRequest` or `fetch()` call, listens for lifecycle events (`load`, `error`, `progress`, `abort`), and translates them into a stream of `HttpEvent` objects pushed to the `Observable`'s subscriber via `observer.next(...)`.
7. `HttpClient`'s outer operators (based on `observe / responseType`) filter/map that raw event stream down to just the body (default), the full `HttpResponse`, or leave it as the raw event stream.

8. On completion, `observer.complete()` fires; on failure, `observer.error(new HttpResponse(...))` fires instead — errors never call `next`, only the error channel.

4.2 Why it's cold and why unsubscribing aborts

The `Observable` constructor callback passed to `HttpClientBackend / FetchBackend` is **not invoked until `subscribe()` is called** — it literally opens the `XMLHttpRequest` (`xhr.open()` + `xhr.send()`) or calls `fetch()` inside that callback. Because RxJS re-runs the subscriber function per subscription, calling `.subscribe()` twice on the same `HttpClient` observable fires **two separate HTTP requests** (this is distinct from a hot/shared multicast subject — `HttpClient` does no caching or sharing on its own; use `shareReplay()` explicitly if you want one shared request across multiple subscribers).

The teardown function returned from that `Observable` constructor calls `xhr.abort()` (or an `AbortController.abort()` for fetch). RxJS invokes this teardown automatically when:

- You call `subscription.unsubscribe()` explicitly.
- The component is destroyed and you used `takeUntilDestroyed() / async` pipe.
- An outer operator like `switchMap / takeuntil / first()` triggers unsubscription of the inner observable.

This is why cancellation "for free" — it's not special-cased `HttpClient` logic, it's the general RxJS teardown contract applied to a subscriber function that happens to wrap a cancellable network call.

4.3 Interceptor chain composition detail

`provideHttpClient(withInterceptors([a, b, c]))` composes interceptors in **array order for the request path**: `a` sees the request first, then hands off to `b`, then `c`, then the real backend. Because interceptors are typically pass-through wrappers around `next(req)`, the *response* path unwinds in reverse — `c`'s response-side logic (e.g., a `tap / catchError` after `next(req)`) runs before `b`'s, which runs before `a`'s. This "onion" model matches Express/Koa middleware and is a common point of confusion in interviews ("does the last interceptor run first or last?" — first on the way out, last on the way back).

4.4 XSRF/CSRF handling

`HttpClient` has built-in XSRF protection: it reads a cookie (default name `XSRF-TOKEN`) and automatically attaches it as a request header (default `X-XSRF-TOKEN`) on same-origin, non-GET/HEAD requests. This is implemented as a built-in interceptor (`HttpXsrfInterceptor`) inserted into the chain automatically — configurable via `withXsrfConfiguration()`, or disabled via `withNoXsrfProtection()`.

5. Edge Cases & Gotchas

1. **Forgetting to subscribe = no request fires at all.** Since the observable is cold, `this.http.post(url, body)` alone does *nothing* — a classic silent bug. "Sequence contains no elements"-style confusion aside, the real symptom is: no network tab entry, no error, nothing. Always `.subscribe()`, pipe through `async`, or use `firstValueFrom / lastValueFrom`.
2. **Double-firing requests.** Subscribing twice to the same `HttpClient` observable (e.g., binding it to two `| async` pipes in a template, or calling `.subscribe()` in a loop) issues the request twice, independently

— each subscription re-runs the underlying XHR/fetch. Fix with `shareReplay({ bufferSize: 1, refCount: true })` if the value should be shared.

3. **Memory leaks from long-lived, unsubscribed requests.** A component that calls `this.http.get(...).subscribe(...)` and gets destroyed before the response arrives leaves a dangling subscription (and, worse, a callback that may try to touch a destroyed component's state, e.g. `this.data = ...` throwing or silently updating a detached view). Fix with `takeUntilDestroyed()` (Angular 16+) or manual `ngonDestroy` unsubscription. Note: unlike a `setInterval`, an HTTP request *does* naturally complete on its own (unless it hangs forever), so the "leak" here is milder than an infinite stream — but it's still wasted work, potential race conditions writing to stale state, and (for large payloads/uploads) real resource waste, so tearing it down is still correct practice.
4. **HttpResponse.error shape is not uniform.** For a server error with a JSON body, `error.error` is the parsed JSON object. For a network failure, CORS block, or timeout, `error.error` is a `ProgressEvent` / `ErrorEvent`-like object with no `.message` in the shape you expect. Code that assumes `error.error.message` always exists will throw or show `undefined` for network failures — always branch on `error.status === 0` first.
5. **Race conditions with concurrent requests — switchMap vs mergeMap vs concatMap.** Using `mergeMap` for a type-ahead search lets old, slow requests resolve *after* newer ones, potentially overwriting the UI with stale results (classic race condition). `switchMap` cancels the previous inner request when a new source value arrives, guaranteeing only the latest search term's response is used. But `switchMap` is wrong for sequential writes (e.g., "save then reload") where you need every request to complete in order — there, `concatMap` is correct, and `exhaustMap` is correct for "ignore new clicks while a save is in flight" (e.g., a submit button).
6. **Retrying non-idempotent requests blindly.** Wrapping a `POST` (e.g., "charge credit card") in `retry(3)` can create duplicate side effects if the first attempt actually succeeded server-side but the response was lost (network blip after the server processed it). Only retry idempotent operations (GET, PUT, DELETE) without additional idempotency-key protection.
7. **204 No Content and empty bodies with responseType: 'json'.** A 204 response has no body; if `responseType` is `'json'` (default) and the body is truly empty, `HttpClient` returns `null` rather than throwing a JSON parse error — but a server that returns 200 with an *empty string* body while claiming `Content-Type: application/json` will cause an actual JSON parse error, surfaced as an `HttpResponse` with `status: 200` and an `error` containing a `SyntaxError`. This is a genuinely confusing case: a 200 that still ends up in your `catchError` block.
8. **HttpParams / HttpHeaders immutability trap.** `params.set('page', '2')` **does not mutate** `params` — it returns a new instance. Code like `params.set('page', '2'); this.http.get(url, { params })` silently sends the *old* params because the return value was discarded. Must reassign: `params = params.set(...)`.
9. **Interceptors and cloning.** Because `HttpRequest` is immutable, forgetting to use `req.clone({...})` and instead trying to mutate `req.headers` directly either fails silently (no effect) or throws depending on the property, since request objects are frozen/read-only by design.
10. **withFetch() and upload progress.** Switching to the fetch backend for SSR benefits can silently degrade upload-progress UX if the app relies on `HttpEventType.UploadProgress` for a progress bar — verify progress events still fire as expected in your target browsers before switching backends app-wide.
11. **Absolute vs relative URLs and base href.** Relative URLs (`/api/users`) resolve against the document's

`<base href>`, not against the currently active route — a common source of confusion in apps with non-root deployments (e.g., an app served from `/my-app/`) if `<base href>` isn't configured correctly.

6. Interview Questions & Answers

Q1. What's the difference between `provideHttpClient()` and `HttpClientModule`?

Both wire up `HttpClient` and its default backend, but `provideHttpClient()` is the standalone-API provider function used with `bootstrapApplication`, composed via `with*()` feature functions (`withInterceptors`, `withFetch`, `withJsonpSupport`, `withXsrfConfiguration`, `withInterceptorsFromDi`). `HttpClientModule` is the legacy `NgModule`-based equivalent, still supported for backward compatibility. `provideHttpClient()` is tree-shakable at a finer grain and is the recommended approach in modern (standalone) Angular apps.

Q2. Does `HttpClient.get<T>()` validate the response body against `T` at runtime?

No. The generic type parameter is a compile-time-only assertion. `HttpClient` parses JSON (producing `any / unknown` structurally) and simply casts it to `T` for TypeScript's benefit — there is no runtime schema validation. If the backend's actual response shape diverges from your interface, you get no error; you just get objects that violate your assumed type silently, which can cause `undefined` property access bugs downstream. Runtime validation requires a library like `zod / io-ts` layered on top, typically inside a `map()` operator after the HTTP call.

Interviewer intent: Tests whether the candidate conflates TypeScript's static type system with actual runtime data validation — a very common misunderstanding among developers who assume "typed HTTP client" implies safety guarantees it doesn't provide.

Q3. Why are `HttpClient` observables described as "cold," and what's the practical consequence?

They're cold because the actual side effect (opening the XHR/fetch call) happens inside the `Observable` constructor's subscriber function, which RxJS only invokes when `subscribe()` is called — and it re-invokes it independently for every new subscription. Practically: (1) an HTTP call you never subscribe to never fires; (2) subscribing twice to the same call object fires two independent HTTP requests, not one shared response; (3) unsubscribing tears down (aborts) that specific request without affecting any other subscription.

Q4. How do you cancel an in-flight HTTP request, and what actually happens under the hood?

Call `.unsubscribe()` on the `Subscription` returned by `.subscribe()` (or let an operator like `switchMap / takeUntil / first()` do it automatically). Internally, `HttpClient`'s backend registered a teardown function when it created the request's `Observable`; that teardown calls `xhr.abort()` (XHR backend) or triggers an `AbortController.abort()` tied to the `fetch()` call (fetch backend), which actually cancels the underlying network request at the browser/OS level rather than merely detaching a callback.

Q5. What's the standard pattern to avoid race conditions in a type-ahead search box, and why?

Pipe the search-term stream through `debounceTime`, `distinctUntilChanged`, then `switchMap` to the HTTP call: `searchTerm$.pipe(debounceTime(300), distinctUntilChanged(), switchMap(term => this.http.get(...))). switchMap` unsubscribes from (and thus cancels/ignores) the previous inner HTTP observable the instant a new outer value arrives, guaranteeing that only the response to the *latest* keystroke ever reaches the subscriber — eliminating the race where a slow response to an earlier query overwrites the UI after a faster response to a later query already rendered.

Interviewer intent: Distinguishes candidates who've actually debugged a "stale search results flashing on screen" bug from those who've only read about the flattening operators in the abstract.

Q6. Explain the difference between `observe: 'body'`, `'response'`, and `'events'`.

`'body'` (default) returns just the parsed response payload typed as `T`. `'response'` returns the full `HttpResponse<T>` wrapper, giving access to `status`, `headers`, and `body` together — needed when you must inspect status codes or response headers (e.g., pagination metadata in a custom header) alongside the payload. `'events'` returns the entire lifecycle as a stream of `HttpEvent<T>` values (`Sent`, `UploadProgress`, `DownloadProgress`, `ResponseHeader`, `Response`, `User`), used for progress bars or fine-grained request lifecycle tracking; it requires switching on `event.type` (typically via `HttpEventType`) to extract the final body.

Q7. How does `HttpErrorResponse` differ for a server error vs. a network-level error, and how should error handling code branch on it?

For a genuine HTTP response with a non-2xx status (e.g., 404, 500), `error.status` is the real status code and `error.error` is the parsed response body (JSON object, if the server sent JSON). For a failure where no HTTP response was ever received — DNS failure, CORS rejection, offline, connection refused — `error.status` is `0` and `error.error` is a browser-level error/progress event object with a very different (and less useful) shape. Robust error-handling code should check `error.status === 0` first to branch into a "network/connectivity" message path before trying to read server-specific fields like `error.error.message`.

Q8. Why must `HttpParams` and `HttpHeaders` updates be reassigned rather than just called?

Both classes are immutable value objects — `.set()`, `.append()`, and `.delete()` all return a **new** instance rather than mutating the receiver in place. This mirrors the immutability of `HttpRequest` itself and is intentional: it keeps request objects safe to pass through an interceptor chain without one interceptor's mutation silently affecting another's view of the same object. The practical consequence is that `params.set('page', '2')` on its own is a no-op from the caller's perspective; the return value must be captured (`params = params.set(...)`) or chained fluently.

Q9. Walk through what happens internally, from `provideHttpClient(withInterceptors([...]))` down to the actual network call, when a component calls `http.get()`.

`HttpClient.get()` builds an immutable `HttpRequest` describing method/url/headers/params/options, then calls the internal `request()` method, which returns a new cold `Observable`. On subscription, the request enters the interceptor chain built from the `withInterceptors([...])` array: each interceptor runs in registration order on the way in, optionally cloning the request (`req.clone({...})`) before calling `next(req)` to forward it, or short-circuiting by returning its own observable (e.g., a cache hit). At the end of the chain sits the real `HttpBackend` — `HttpXhrBackend` or `FetchBackend` depending on whether `withFetch()` was configured — which performs the actual `XMLHttpRequest` / `fetch()` call and translates its native lifecycle events into a stream of `HttpEvent` objects. `HttpClient`'s outer logic then filters/maps that stream according to `observe / responseType` before delivering to the caller's subscriber. Responses (and any interceptor logic layered *after* their own `next(req)` call) unwind back through the chain in reverse order, like middleware.

Interviewer intent: Checks for a genuine mental model of `HttpClient` as a composed `Observable` pipeline — not just memorized API surface — and whether the candidate understands ordering semantics of interceptors, which is a frequent source of real bugs (e.g., an auth interceptor placed after a logging interceptor never gets its token attached before the log line fires).

Q10. How would you unit test a service method that calls `HttpClient`, without hitting a real network?

Configure the `TestBed` with `provideHttpClient()` and `provideHttpClientTesting()` (or `HttpClientTestingModule` in module-based tests), inject `HttpTestingController` alongside the service under test, call the service method and subscribe with assertions in the callback, then use `httpMock.expectOne(urlOrMatcherFn)` to retrieve the pending `TestRequest`, assert on `req.request.method / .headers / .params / .body`, and resolve it synchronously with `req.flush(mockBody, {`

`status, statusText }`) (or `req.error(...)` to simulate a network failure). Always call `httpMock.verify()` in `afterEach` to catch any request that was made but never asserted/flushed, or expected but never made.

Q11. What's the difference between `retry()` and adding your own manual retry with `catchError` + recursion, and when is blind `retry()` dangerous?

`retry({ count, delay })` is a built-in RxJS operator that resubscribes to the source observable (re-issuing the HTTP request) up to `count` times on error, optionally with a delay or backoff strategy (`retry({ count, delay: (error, retryCount) => timer(retryCount * 1000) })`). It's declarative and composes cleanly in a `.pipe()` . It's dangerous for non-idempotent operations (`POST` that creates a resource, charges a payment, sends an email) because a "failure" might mean the server processed the request successfully but the response was lost in transit — blindly retrying can duplicate the side effect. Idempotent methods (`GET` , `PUT` , `DELETE` on a specific resource) are safe to retry by default; `POST` should only be retried if the backend supports idempotency keys or the operation is provably safe to repeat.

Q12. Why does calling `.subscribe()` twice on the same `this.http.get(url)` call object result in two separate network requests instead of one shared response?

Because `HttpClient` observables are cold and unshared by default — no multicasting happens automatically. Each `.subscribe()` call independently re-invokes the underlying subscriber logic that opens a fresh `XMLHttpRequest` / `fetch()` call. This is standard cold-observable behavior, not an `HttpClient`-specific quirk. To share a single response across multiple subscribers (e.g., a value bound to multiple template bindings or consumed by multiple components), you must explicitly apply a multicasting operator like `shareReplay({ bufferSize: 1, refCount: true })` , which caches the latest emission and replays it to late subscribers instead of re-triggering the source.

Q13. What is `HttpXsrfInterceptor` and how does Angular's built-in CSRF protection work by default?

Angular ships a built-in interceptor that reads a cookie (default name `XSRF-TOKEN` , typically set by the server) and automatically attaches its value as a request header (default `X-XSRF-TOKEN`) on outgoing same-origin, state-changing requests (non-`GET`/`HEAD`). This implements the double-submit cookie pattern for CSRF protection with zero application code required, as long as the backend sets the cookie and validates the header matches it. It's configurable via `withXsrfConfiguration({ cookieName, headerName })` or disabled entirely via `withNoXsrfProtection()` if the backend uses a different CSRF strategy.

Interviewer intent: Distinguishes candidates who know `HttpClient` has *built-in* security plumbing (often overlooked since it "just works") from those who'd reach for a manual interceptor to solve a problem Angular already solves, or who don't realize why a CSRF token header shows up in the network tab without any code writing it.

Q14. How does the `fetch()`-based backend (`withFetch()`) differ from the default XHR backend, and when would you choose it?

`withFetch()` swaps `HttpXhrBackend` for `FetchBackend` , implementing the same `HttpBackend` interface but built on the native `fetch()` API instead of `XMLHttpRequest` . Benefits: works natively in server-side/Node environments during SSR without needing an XHR polyfill, smaller/more modern surface, and better alignment with streaming response bodies. Trade-off: upload progress reporting is less reliable/consistent across browsers with `fetch()` than with XHR's `upload.onprogress` , so an app with a critical file-upload progress bar should verify (or stick with the XHR backend) before switching globally. Both backends are drop-in swappable via provider configuration — no application/service code changes required.

7. Quick Revision Cheat Sheet

- **Setup:** `provideHttpClient(withInterceptors([...]), withFetch())` (standalone) or `HttpClientModule` (NgModule/legacy).
- **Methods:** `.get<T>()`, `.post<T>()`, `.put<T>()`, `.patch<T>()`, `.delete<T>()` — all return `Observable<T>`; generics are compile-time only, **no runtime validation**.
- **HttpParams / HttpHeaders:** immutable — `.set()` returns a new instance; must reassign.
- **observe option:** `'body'` (default, just payload) / `'response'` (full `HttpResponse<T>`: status + headers + body) / `'events'` (full lifecycle stream, needs `reportProgress: true` for progress events).
- **Progress tracking:** build an `HttpRequest` with `reportProgress: true`, call `http.request(req)`, switch on `HttpEventType` (`Sent`, `UploadProgress`, `DownloadProgress`, `ResponseHeader`, `Response`).
- **Errors:** land on the error channel as `HttpErrorResponse`; `status === 0` $\Rightarrow$ network/CORS/offline (no real HTTP response); non-zero $\Rightarrow$ real server status, `error.error` = parsed body. Use `catchError` + `throwError(() => ...)`.
- **Cancellation:** `.unsubscribe()` calls `xhr.abort()` / `AbortController.abort()` under the hood — real cancellation, not just detaching a listener. `switchMap` is the idiomatic "cancel previous, use latest" pattern.
- **Cold observables:** no `.subscribe()` $\Rightarrow$ no request ever fires; two `.subscribe()`s $\Rightarrow$ two independent requests; use `shareReplay()` to share one response.
- **Interceptors:** functional (`HttpInterceptorFn`) registered via `withInterceptors([...])`, run in array order inbound, reverse order outbound (onion/middleware model); must `req.clone({...})` since requests are immutable.
- **Backends:** `HttpXhrBackend` (default, XHR-based, reliable upload progress) vs `FetchBackend` (`withFetch()`, native fetch, better for SSR, weaker upload-progress support).
- **XSRF:** built-in `HttpXsrfInterceptor` auto-attaches `X-XSRF-TOKEN` header from `XSRF-TOKEN` cookie on non-GET requests; configurable/disable-able.
- **Testing:** `provideHttpClientTesting()` + `HttpTestingController` $\rightarrow$ `expectOne()`, `req.flush(body, opts)`, `req.error(...)`, always `httpMock.verify()` in `afterEach`.
- **Gotchas to remember:** forgotten subscribe = silent no-op; unsubscribed long-lived requests risk stale-state writes; 200 status with malformed/empty JSON body still throws into `catchError`; blind `retry()` on non-idempotent `POST` can duplicate side effects; `mergeMap` in search boxes causes race conditions, `switchMap` fixes them, `concatMap` / `exhaustMap` are correct for ordered writes/submit-button debouncing respectively.

Created By - Durgesh Singh